26-SPRING 지능형 로보틱스 (01)

CookMate Arm

2371069 Hwang Youngseo

Department of Computer Science and Engineering

Outline

1. Project Goal and Topic
2. System Architecture
3. Project Details
4. Implementation & Demo
5. Future Work and Improvement

01

Project Goal and Topic

Project Goal and Topic

Project Topic

- Implementation of a **Pick-and-Place System** Using a Panda Robot in ROS2 and Gazebo Simulation Environment

Project Goal

- Implementation of a task in which the robot **picks up a selected object and places it into a pot** for helping user's cooking

- To achieve this, a robot control environment was built based on **ROS2 Humble**. **MoveIt2** and **Gazebo** were integrated to control the robot arm and gripper.
- A **custom ROS2 node** was also created to execute the overall sequence, including moving the robot arm, opening and closing the gripper, approaching the object, picking it up, and placing it at the target position.

Project Goal and Topic

Initial Plan

- Use the existing Pick-and-Place branch
- Implement a separate planning node, perception node with logical camera
- Use the planning node to control object picking and placing automatically
- Build a more detailed workspace with multiple objects

Problem

- The Pick-and-Place branch did not run properly in my environment
- Because of this issue, the original implementation plan had to be changed
- Due to the hardware limitations, the workspace was simplified to include only the essential objects

Final Implementation

- Implemented a custom **pick_object_node**
- Used **hard-coded object and target positions**
- **Planning node** is changed into directly specified **through commands**

- **Focused on verifying the Pick-and-Place sequence first**

Project Goal and Topic

Initial Plan

- Use the existing Pick-and-Place branch
- Implement a separate planning node, perception node with logical camera
- Use the planning node to control object picking and placing automatically
- Build a more detailed workspace with multiple objects

Problem

- The Pick-and-Place branch did not run properly in my environment
- Because of this issue, the original implementation plan had to be changed
- Due to the hardware limitations, the workspace was simplified to include only the essential objects

Final Implementation

- Implemented a custom **pick_object_node**
- Used **hard-coded object and target positions**
- **Planning node** is changed into directly specified **through commands**

Focused on verifying the Pick-and-Place sequence first

02

System Architecture

Project Architecture

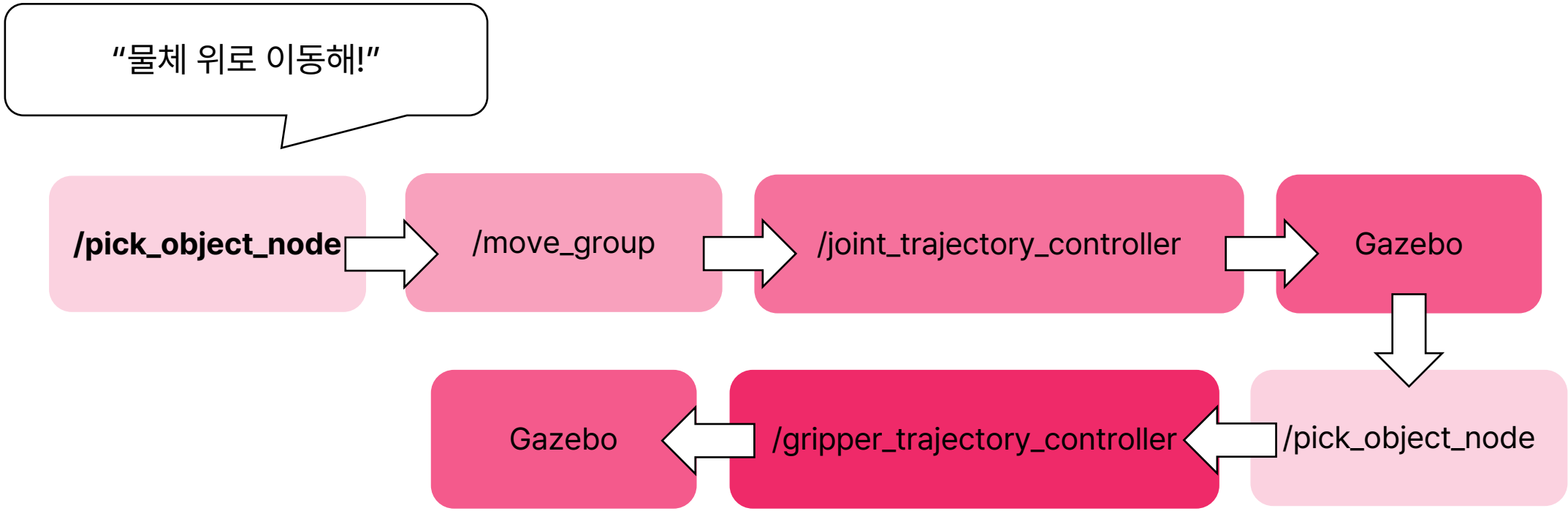
/pick_object_node

- This node controls the overall Pick-and-Place sequence.
  It sends commands to move the Panda robot arm and control the gripper.

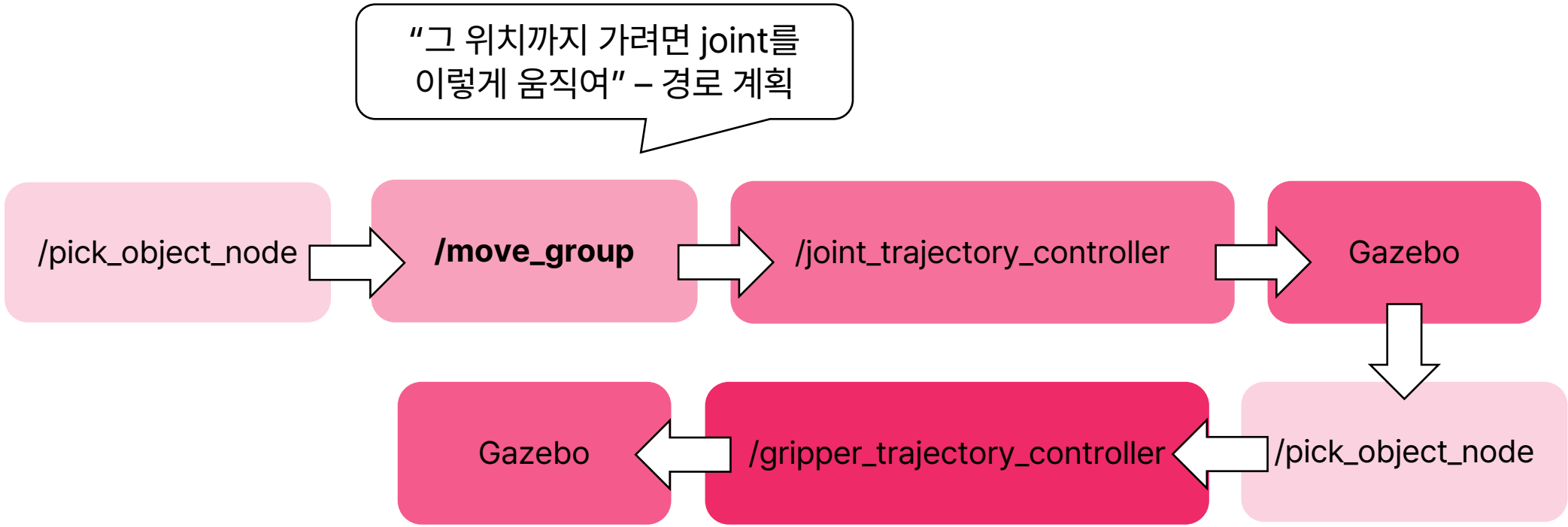
Existing Nodes Used in the System

Node	Role
/move_group	plans the motion path of the robot
/joint_trajectory_controller	executes the arm movement
/gripper_trajectory_controller	opens and closes the gripper
/controller_manager	manages the robot controllers
/robot_state_publisher	publishes the robot structure
/joint_state_broadcaster	publish the current joint states
Gazebo/GZ	simulates the robot, objects, and environment

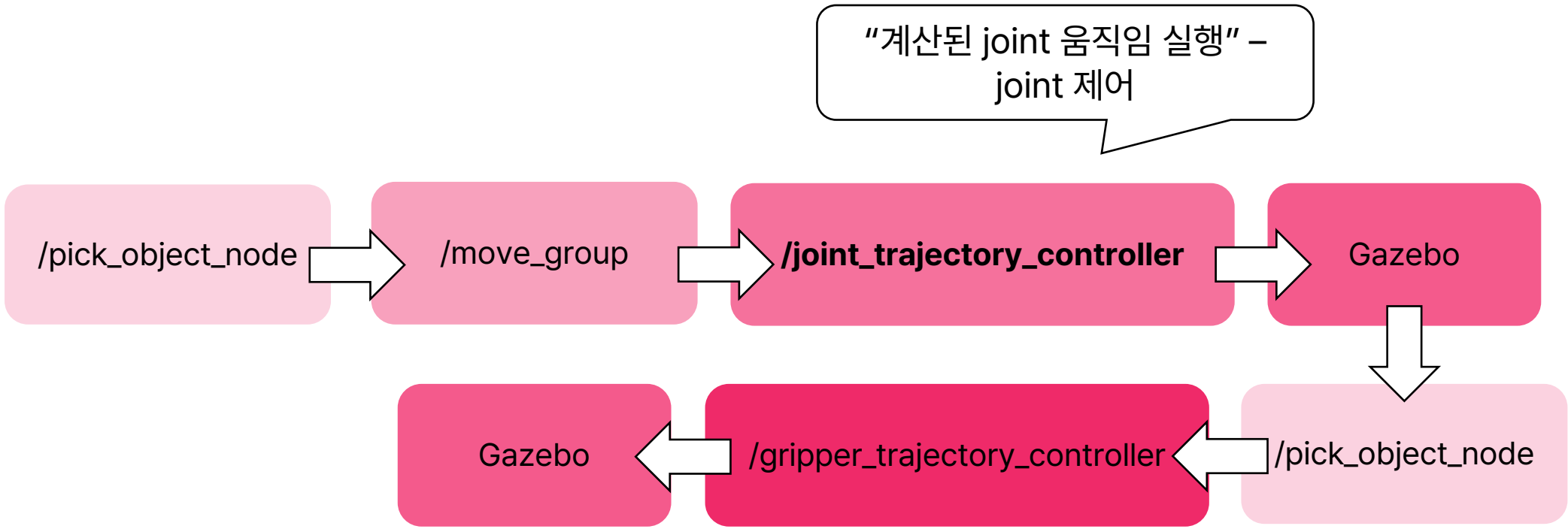
pick_object_node



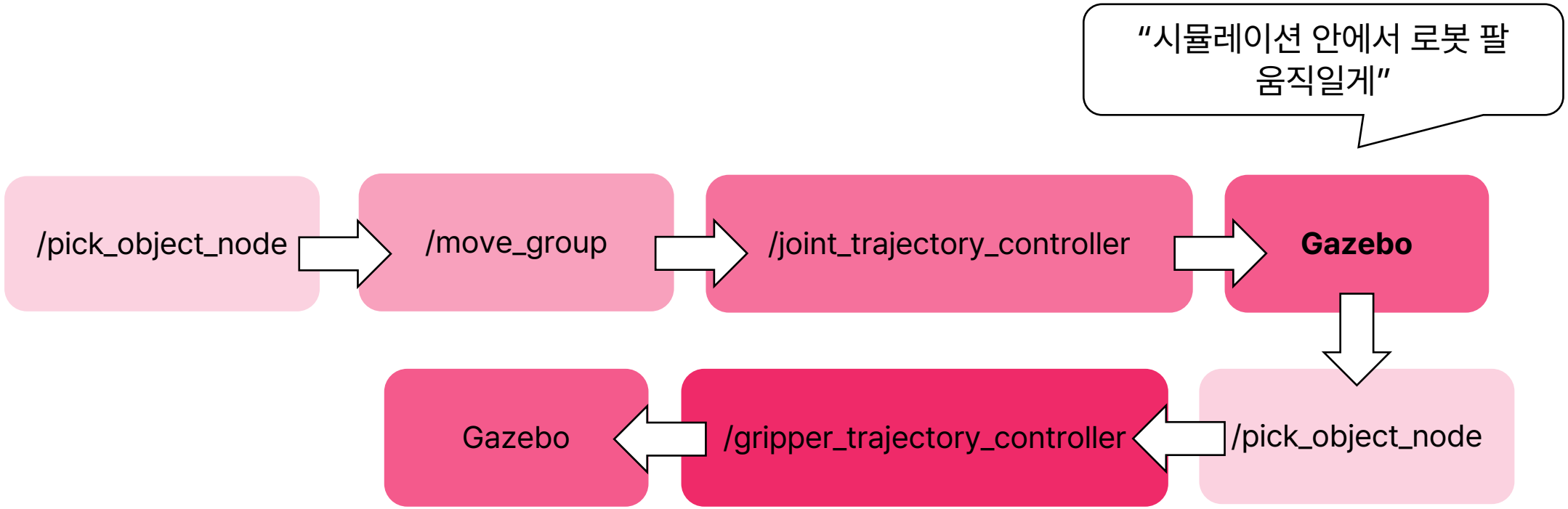
Project Architecture



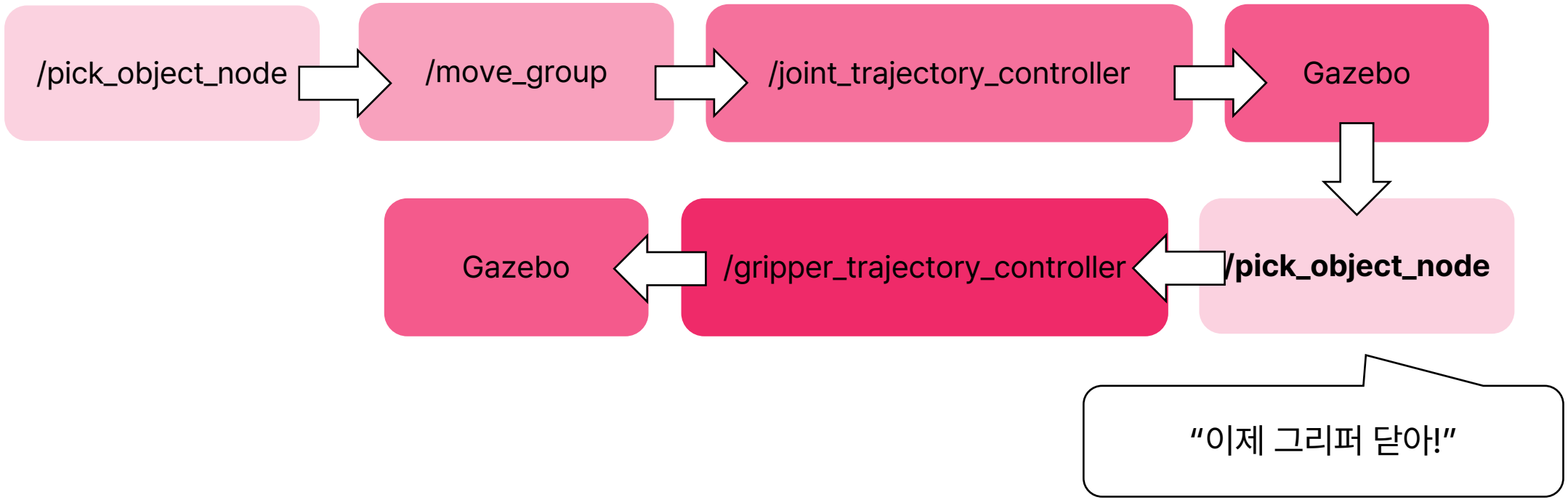
Project Architecture



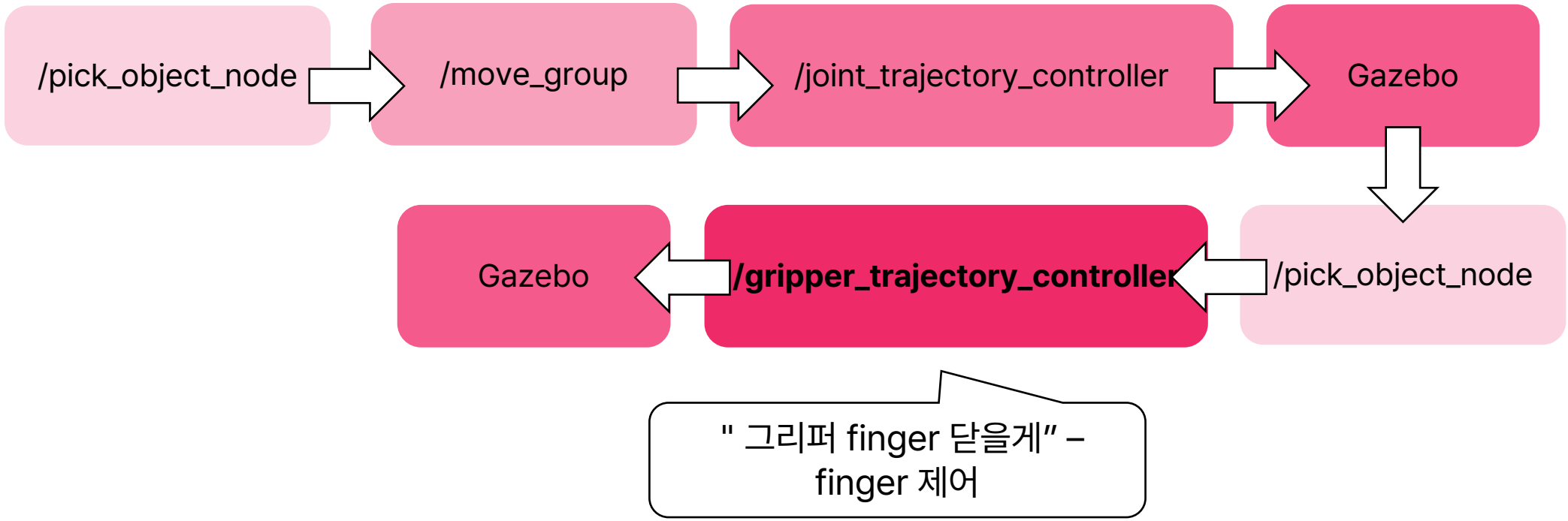
Project Architecture



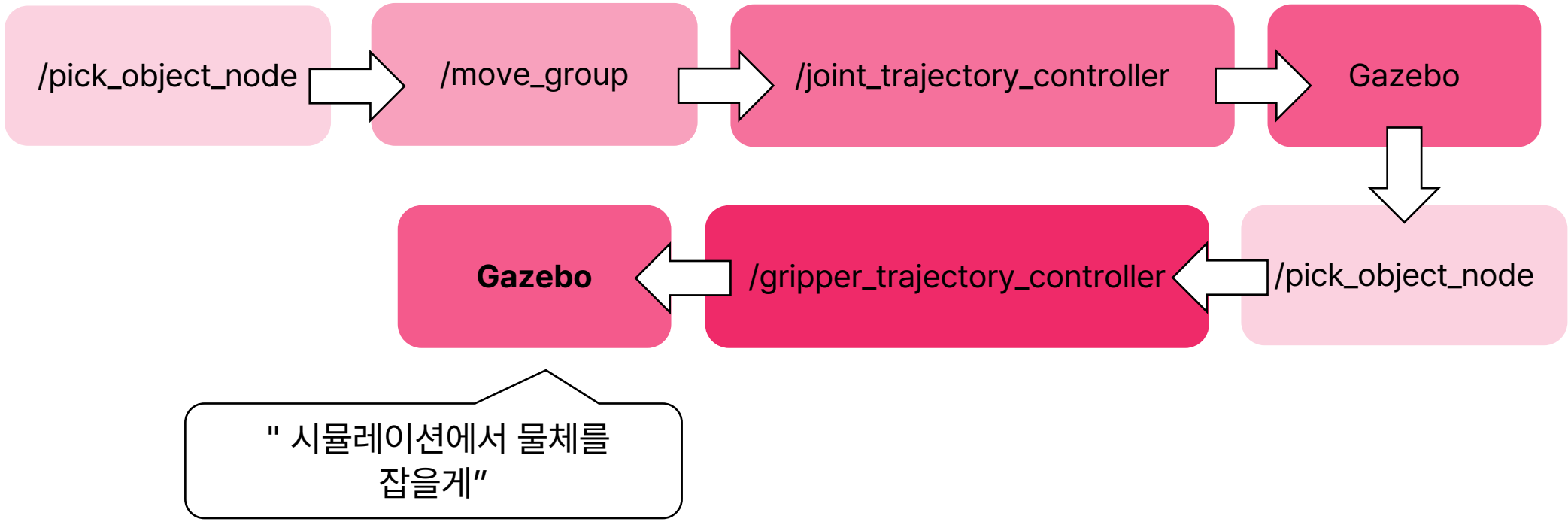
Project Architecture



Project Architecture



Project Architecture



03

Project Details

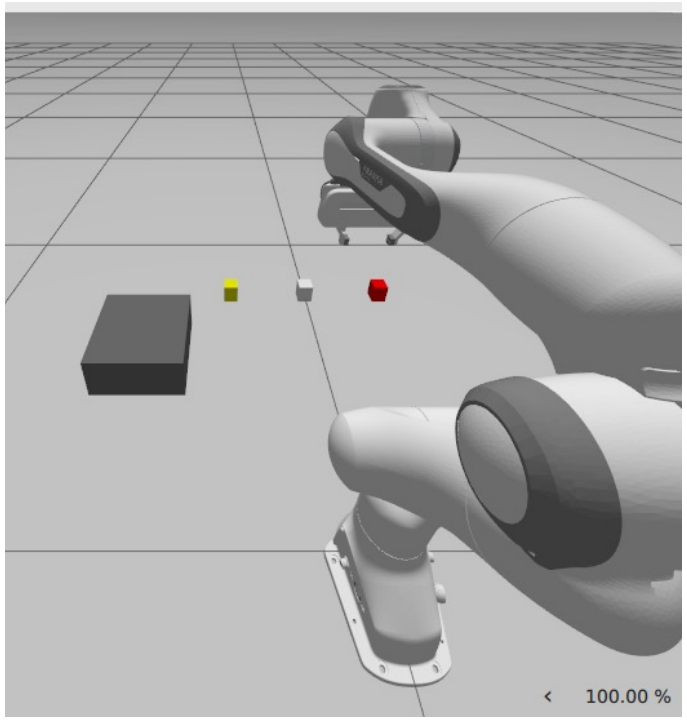
Project Details

Development Environment

Software/Tool	Version/Branch
ROS2	Humble
Gazebo	Fortress
panda_gz_moveit2	humble_devel branch

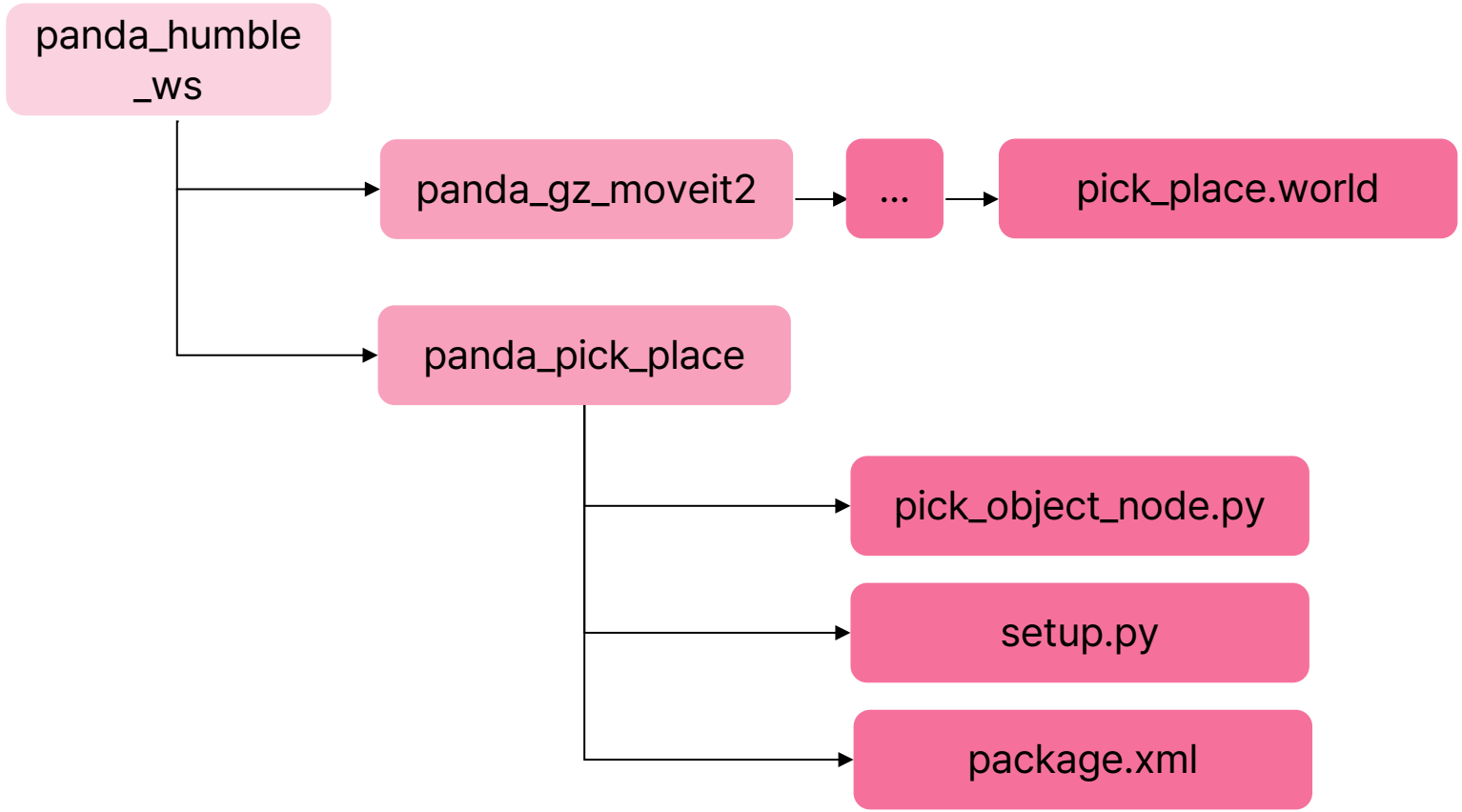
Workspace Configuration

- panda robot arm
- Example objects
  - Red box (ham)
  - White box (sugar)
  - Yellow box (cheese)
  - box-shaped object (pot)



Project Details

Package Structure



Project Details

Object Coordinate Definition

pick_object_node.py

```
OBJECT_POSES = {
    # ham = red box
    'ham': {
        'model_name': 'red_box',
        'x': 0.71,
        'y': -0.25,
        'z': 0.03,
        'pick_offset_z': 0.025,
        'grip_width': 0.008,
    },
    # sugar = white box
    'sugar': {
        'model_name': 'white_box',
        'x': 0.71,
        'y': 0.00,
        'z': 0.03,
        'pick_offset_z': 0.03,
        'grip_width': 0.008,
    },
    # cheese = yellow box
    'cheese': {
        'model_name': 'yellow_box',
        'x': 0.66,
        'y': 0.28,
        'z': 0.03,
        'pick_offset_z': 0.035,
        'grip_width': 0.008,
    },
}
```

parameter	meaning
x	Object position along the x-axis in the Gazebo world
y	Object position along the y-axis in the Gazebo world
z	Object height or base position in the Gazebo world
pick_offset_z	Height offset added to the object z-coordinate for grasp ing
grip_width	Gripper closing width used to grasp the object

Pick-and-Place Motion Implementation

Motion Sequence

1. Open the gripper

```
Step 0: open gripper
```

2. Move above the object

```
Step 1: move above ham
Sending MoveIt goal: x=0.710, y=-0.250, z=0.210
MoveIt goal accepted.
```

3. Approach the object

```
Step 2: move down to grasp ham
Sending MoveIt goal: x=0.710, y=-0.250, z=0.055
MoveIt goal accepted.
```

4. Close the gripper

```
Step 3: close gripper around ham
```

5. Lift the object

```
Step 4: lift ham
Sending MoveIt goal: x=0.710, y=-0.250, z=0.250
MoveIt goal accepted.
```

6. Move to the target position

```
Step 5: move above pot
Sending MoveIt goal: x=0.450, y=0.500, z=0.350
MoveIt goal accepted.
```

7. Move down to the target position

```
Step 6: move down to place position
Sending MoveIt goal: x=0.450, y=0.500, z=0.180
MoveIt goal accepted.
```

8. Open the gripper

```
Step 7: open gripper to release ham
```

9. Return to a safe position

```
Step 8: lift away from pot
Sending MoveIt goal: x=0.450, y=0.500, z=0.350
MoveIt goal accepted.
```

Project Details

Motion Sequence Code

```
def run_sequence(self):
    if self.move_in_progress:
        return

    obj_x = self.obj_x
    obj_y = self.obj_y
    obj_z = self.obj_z

    pot_x = self.pot_x
    pot_y = self.pot_y

    if self.current_step == 0:
        self.get_logger().info('Step 0: open gripper')
        self.move_gripper(0.035)
        self.current_step += 1

    elif self.current_step == 1:
        self.get_logger().info(f'Step 1: move above {self.target}')
        self.send_moveit_pose_goal(obj_x, obj_y, obj_z + 0.18)

    elif self.current_step == 2:
        self.get_logger().info(f'Step 2: move down to grasp {self.target}')
        self.send_moveit_pose_goal(obj_x, obj_y, obj_z + self.pick_offset_z)
```

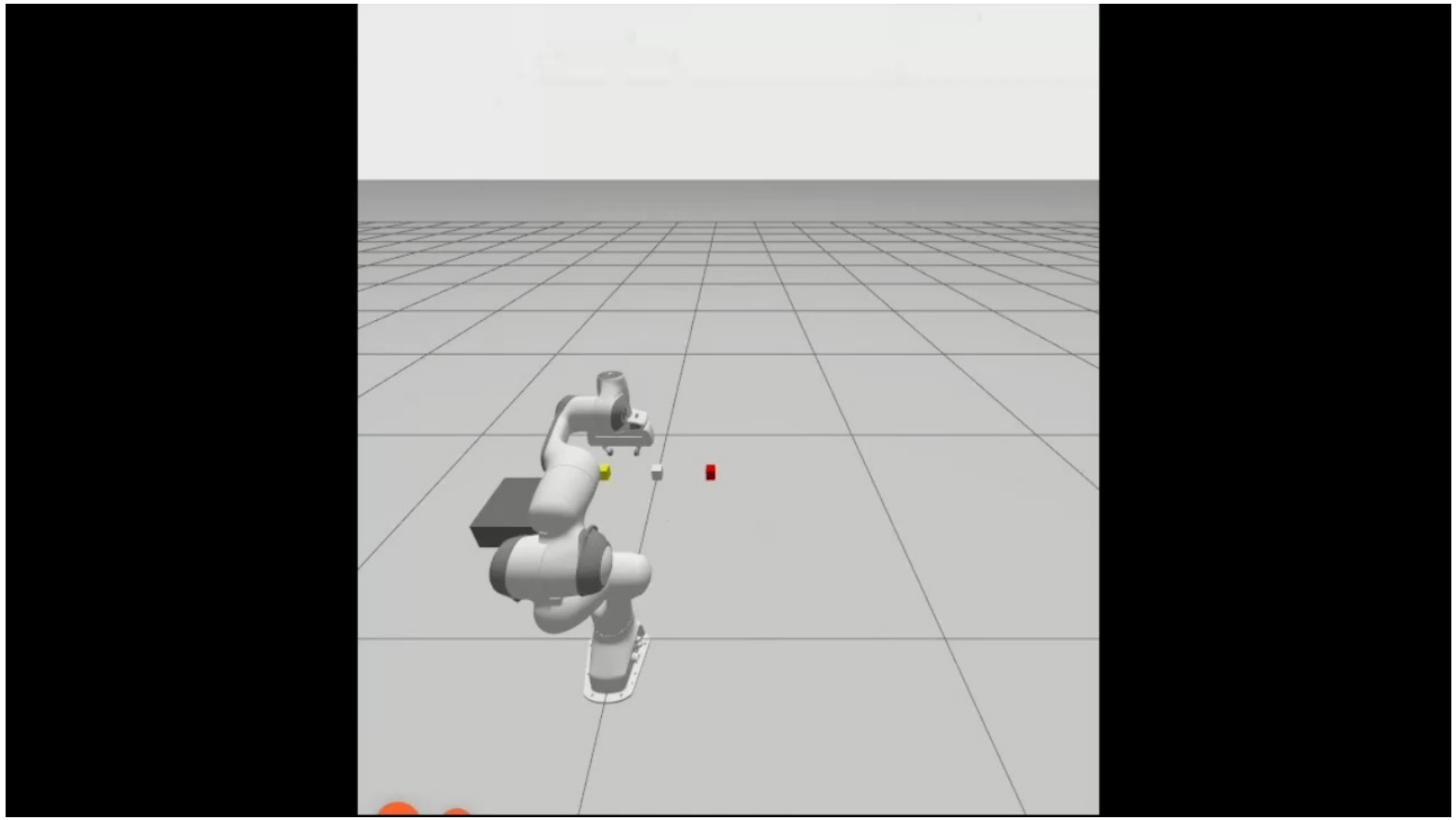
04

Implementation and Demo

Implementation and Demo

- Grip ham (red_box)

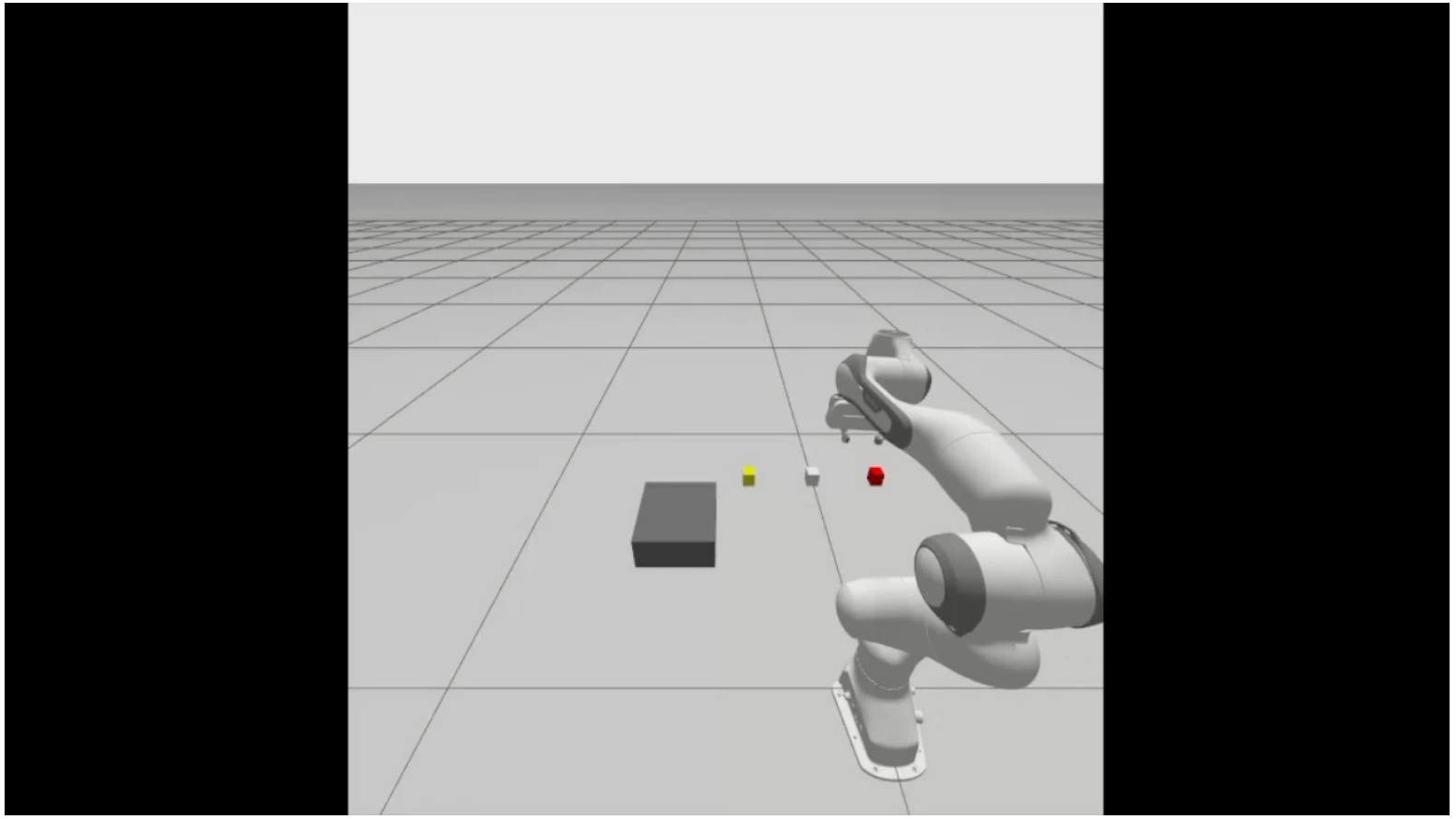
```
youngseo@youngseo:~/panda_humble_ws$ ros2 run panda_pick_place pick_object_node --ros-args -p target:=ham
```



Implementation and Demo

- Grip cheese (yellow_box)

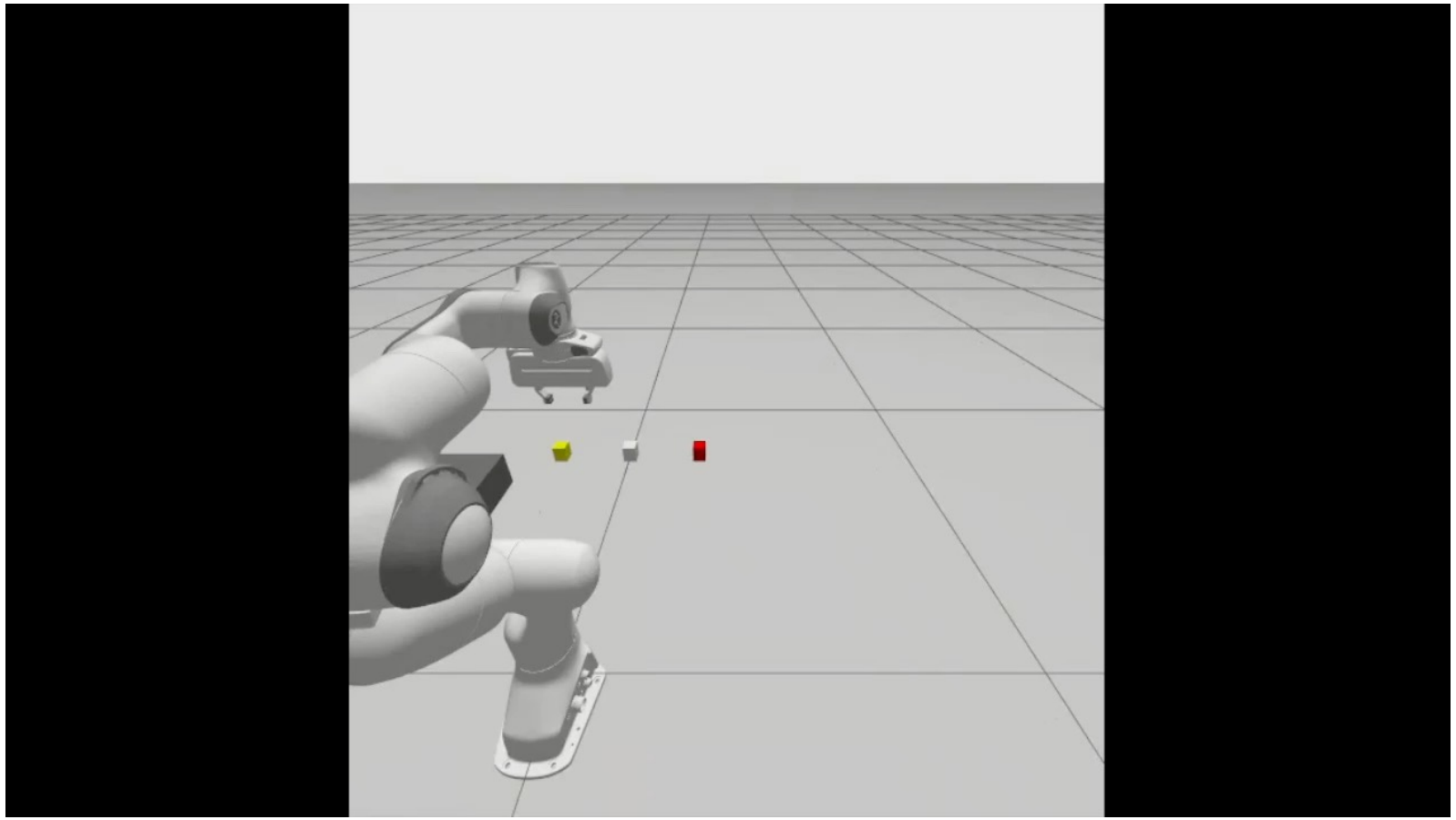
```
youngseo@youngseo:~/panda_humble_ws$ ros2 run panda_pick_place pick_object_node --ros-args -p target:=cheese
```



Implementation and Demo

- Grip sugar (white_box)

```
youngseo@youngseo:~/panda_humble_ws$ ros2 run panda_pick_place pick_object_node --ros-args -p target:=sugar
```



05

Future Work and Improvement

Future Work and Improvement

Sensor-Based Object Detection

- Current method: hard-coded coordinates
- Future Improvement: detected by camera or sensor

More Accurate Vertical Descent Motion

- move above object → descend vertically → close gripper → lift vertically

Automatic Planning Node

- Implement a separate planning node to decide the object, target position, and motion sequence automatically.

Thank You :)

EWHA WOMANS UNIVERSITY